Static & Final in Java

Static Variables, Static Methods, Static Block, Final, and `String[] args`

1. The Problem: The Same Value Copied Into Every Object

Suppose every student in your app studies at the **same college**.

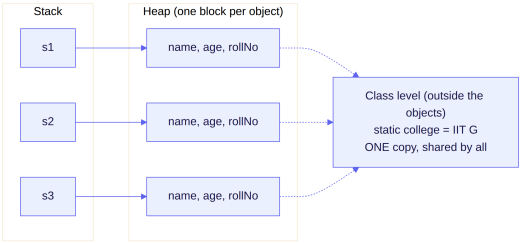
```
class Student {
    String name;
    int age;
    int rollNo;
    String college;   // same value for every student!
}
```

With 1000 `Student` objects, every one stores its own copy of `college` (a 4-byte reference each), all holding the same text that nobody will ever change. That is wasted space.

The fix: keep **one** copy outside the objects and let every object **share** it. The `static` keyword does exactly this.

2. Static Variables (Class Variables)

```
class Student {
    String name;            // instance variable (one per object)
    int age;
    int rollNo;
    static String college;  // static variable (ONE for the whole class)
}
```



	Instance variable	Static variable
Belongs to	One **object**	The whole **class**
Copies	One per object	**One** in total
Also called	Object variable	**Class variable**
Stored	Inside each object in the heap	**Outside** the objects (not in the object's heap block)
Example	`name`, `age`, `rollNo`	`college`

How to use it

```
Student s1 = new Student();
s1.college = "IIT G";          // works, but gives a compiler warning
```

```
Student.college = "IIT G";      // recommended: ClassName.variableName
```

Syntax: `ClassName.variableName = value;`

📌 **One copy means one change affects everyone.** If you write `s3.college = "IIT Kharagpur";`, then `s1.college` and `s2.college` also print `IIT Kharagpur`, because they all read the same single variable.

⭐ **Exam Tip:** Using an object (`s1.college`) compiles but warns *"The static field should be accessed in a static way"*. It is a warning, not an error. Prefer the class name.

💡 **Hinglish:** *Jo value sabhi objects ke liye same hai (jaise college ka naam), use har object mein copy nahi karte. Ek baar class level pe rakhte hain, aur sab share karte hain.*

---

3. Static Methods

`static` can also go in front of a method. The method then belongs to the **class**, not to any object.

```
class Student {
    static void markAttendance() { ... }
}

Student s1 = new Student();
s1.markAttendance();          // works (warning)
Student.markAttendance();     // recommended, works even with NO object created
```

📌 Methods (static **or** normal) are **never stored inside an object**. An object only holds its own instance variables. All objects share the same single copy of a method's code.

The 3 Rules of Static Methods

#	Rule	Why
1	A static method can only call **other static methods**	You can call it with no object. If it called a normal method, Java wouldn't know *which object's* method to run
2	A static method can only use **static variables**	With no object created, `name`, `age`, `rollNo` don't even exist yet
3	A static method **cannot use `this`**	`this` means "the current object", and a static method has no current object

```
class Demo {
    int x = 5;                  // instance variable
    static int y = 10;          // static variable

    static void show() {
        System.out.println(y);       // OK
        System.out.println(x);       // ERROR: cannot use instance variable in
static context
        System.out.println(this.x);  // ERROR: no 'this' in a static method
    }
}
```

📌 **The reverse is fine.** A normal (instance) method **can** use static variables and call static methods.

---

4. Static Block

A static block is code that **initializes static variables**. A constructor does this job for instance variables, but static variables belong to the class, so a constructor is the wrong place.

```
        class Student {
            String name;
            int age;
            int rollNumber;
            static String college;
            static String Location;

            Student(String name, int age, int rollNumber) {
                this.name = name;
                this.age = age;
                this.rollNumber = rollNumber;
            }

            static {                                    // static block
                college = "Noida International University";
                Location = "Greater Noida";
            }
        }
```

```
        Student s1 = new Student("Md Rounak", 21, 108);
        Student s2 = new Student("Syed Abbas", 22, 109);
        System.out.println(s1.name + ", " + s1.age + ", " + s1.rollNumber + ", " +
Student.college + ", " + Student.Location);
        System.out.println(s2.name + ", " + s2.age + ", " + s2.rollNumber + ", " +
Student.college + ", " + Student.Location);
```

Output:

```
Md Rounak, 21, 108, Noida International University, Greater Noida
Syed Abbas, 22, 109, Noida International University, Greater Noida
```

Fact	Detail
When it runs	**Once**, when the class is loaded, **before any object is created** (and before `main` runs)
Used for	Giving values to static variables (as many as you like in one block)
Alternative	Assign at the declaration: `static String college = "NIU";`

⚠ **Common Mistake:** Assigning a static variable in a constructor (`this.college = college;`) warns *"static field should be accessed in a static way"*, and would **reset it for every object** each time one is created. (`college = college;` just assigns the parameter to itself.) Use a static block or the declaration instead.

---

5. What Can Be `static` ?

Thing	Can be static?	Reason
Variables (fields)	✅ Yes	Shared by the whole class
Methods	✅ Yes	Belong to the class
Block	✅ Yes	Runs once at class loading

Thing	Can be static?	Reason
Method parameters	❌ No	A parameter is a **local variable** (stack) that vanishes when the method ends, so class-level makes no sense
A normal (top-level) class	❌ No	Only a **nested class** (a class inside a class) can be static. Nested classes come later

6. The `final` Keyword

> **Definition:** `final` makes a variable a **constant**. Once it has a value, nobody can change it.

```
class Random {
    final double PI = 3.14;
}

Random r1 = new Random();
System.out.println(r1.PI);   // 3.14
r1.PI = 3.16;                // ERROR: the final field Random.PI cannot be assigned
```

Why use it? Pi never changes. You don't want anyone passing a wrong value or editing it later.

Naming convention: constants are written in **UPPER_CASE**, with underscores between words: `PI`, `PI_VALUE`, `MAX_SPEED`.

When must a final variable get its value?

- **Final instance field:** at declaration, **or** once in the constructor (`final double PI;` then `this.PI = 3.14;`).
- **Final local variable:** `final int x = 4;`, or declare first and assign **exactly once** (`final int x; x = 4;`).
- A **second** assignment is a compile error: *"The final local variable x may already have been assigned"*.

`final` can also go on parameters, methods and classes

- **Parameter:** the method cannot change it (`void print(final String name)`).
- **Method / class:** a different meaning, explained later with inheritance and polymorphism.

---|---| | **Variable** | Becomes a constant | | **Parameter** | The method cannot change that parameter's value (`void print(final String name)`) | | **Method** | Different meaning, explained later with inheritance/polymorphism | | **Class** | Different meaning, explained later with inheritance |

7. `static final` Together

A constant that is also the **same for every object** is written `static final`. This is the standard way to make a class-wide constant.

```
class Random {
    static final double PI;
    static {
        PI = 3.14;        // a static final must be set here or at its declaration
    }
}
// Random.PI  -> 3.14   (access through the class name)
```

⚠ **Common Mistake:** You **cannot** assign a `static final` variable in a constructor. A constructor runs per object, but a static variable belongs to the class. Java reports *"The blank final field PI may not have been initialized"*. Use the declaration (`static final double PI = 3.14;`) or a static block.

8. Why Is `main` Static?

```
public static void main(String[] args) { ... }
```

The JVM starts your program by running `main`. If `main` were not static, the JVM would first need an object (`Demo4 d1 = new Demo4();`) and then `d1.main()` , but it can't build your objects for you. Because `main` is static, the JVM simply calls `Demo4.main()` . No object needed.

📌 So every method in the same class that you call **directly from** `main` must also be `static` , or you get *"Cannot make a static reference to the non-static method"*. (`public` is explained later.)

9. `String[] args` : Command-Line Arguments

`args` is an array of Strings holding anything you type **after the class name** when you run the program.

```
public class Demo5 {
    public static void main(String[] args) {
        System.out.println("Number of arguments are " + args.length);
        for (int i = 0; i < args.length; i++) {
            System.out.println("Argument " + i + " = " + args[i]);
        }
    }
}
```

Command in the terminal	Output
`javac Demo5.java` (compile once)	creates `Demo5.class`
`java Demo5`	`Number of arguments are 0`

`java Demo5 Aditya Tandon Rohit Negi`	`Number of arguments are 4`
	`Argument 0 = Aditya`
	`Argument 1 = Tandon`
	`Argument 2 = Rohit`
	`Argument 3 = Negi`

Uses: passing settings at start-up, e.g. `java Demo5 input.txt output.txt` (read this file, write that file). Rare in small programs today, but frameworks such as Spring Boot use it for start-up configuration.

🌍 Real-Life Examples

Concept	Real-world picture
Static variable	The college or company name printed on every ID card. One value, shared by thousands of people
Static method	`Math.sqrt(25)` and `Integer.parseInt("12")` need no object, you just call them through the class

Concept	Real-world picture
`final` / `static final`	`Math.PI`, a GST rate, `MAX_LOGIN_ATTEMPTS = 3`, or an app version number. Fixed, shared, never edited by mistake
Command-line args	Starting a server with `java -jar app.jar --port=8080`. The setting arrives in `args`

⭐ One-Page Revision

- **Static** = belongs to the **class**, one shared copy. Works on variables, methods and blocks (not parameters, not top-level classes). Access with `ClassName.member`.
- **Static method rules:** calls only static methods, uses only static variables, no `this`.
- **Static block:** runs **once**, at class loading, before any object and before `main`.
- `final` = constant, assigned once, named in `UPPER_CASE`. `static final` = shared constant (set at declaration or in a static block, not in a constructor).
- `main` **is static** so the JVM can run it without an object. `args` holds the words typed after the class name.

📝 Practice Questions

Concept questions

1. What is the difference between an instance variable and a static variable?
2. Why can't a static method use `this` or an instance variable?
3. Why is `main` declared `static`?
4. What is the difference between `final` and `static final`?

Programming questions

1. Create a class `Employee` with fields `name`, `id` and a **static** `company` set to `"TechCorp"` in a **static block**. Create 3 employees, print all details, then change `company` through one object and print again. What do you observe?
2. Add a `static int count` to a `Student` class. Increase it in the constructor. Create 4 students and print `Student.count`.
3. **Predict the output:**

```
class Test {
    static { System.out.println("A"); }
    Test() { System.out.println("B"); }
    public static void main(String[] a) {
        System.out.println("C");
        new Test();
        new Test();
    }
}
```

4. Which lines give an error, and why?

```
final int a = 5;  a = 6;
final int b;      b = 7;
final int c;      c = 1;  c = 2;
```

5. Print the number of command-line arguments, then the arguments in **reverse order**. If there are none, print `No arguments given`.